# Graph traversal and connectivity

Move from BFS distance layers and DFS time intervals to dependency order, strongly connected components, and single-vertex or single-edge failures.

Study **BFS → DFS → topological sorting → SCCs → low, articulation points, bridges, and biconnected blocks**.

Prerequisites are queues, stacks, recursion, sets, and induction. Fix the graph model and adjacency order before tracing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Choose traversal information from the required output

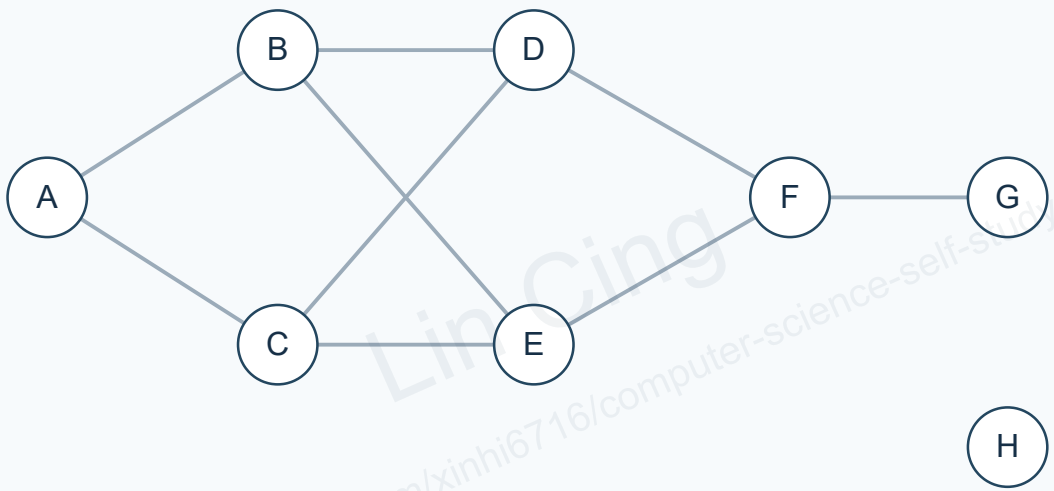
| Question | Information retained | Proof mechanism |
|---|---|---|
| Fewest edges to a vertex? | BFS distances and parents | FIFO layer invariant |
| When does a visit start and finish? | DFS colors, forest, times | Parenthesis and white-path theorems |
| Which task may run first? | DAG finish order or indegrees | Every arc respects the order |
| Which vertices reach each other? | SCCs and condensation DAG | Finish times and component boundaries |
| Does a deletion disconnect the graph? | Undirected low and edge stack | Subtree escape to an ancestor |

BFS d denotes distance; DFS d denotes discovery time. Keep SCC lowlink updates distinct from undirected low updates.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study/

# The graph model determines how edges count

Fixed graph G, source A, isolated H




8 vertices, 9 undirected edges. Neighbors in alphabetical order.

Unweighted simple graph: V={A,B,C,D,E,F,G,H}; edges {AB,AC,BD,BE,CD,CE,DF,EF,FG}. From A, find each minimum edge distance and a corresponding path. H is isolated.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Lists and a matrix store the same adjacency relation

Two representations of the same graph

Adjacency lists

A: B, C

B: A, D, E

C: A, D, E

D: B, C, F

E: B, C, F

F: D, E, G

G: F

H: ∅

| | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 | 0 | 0 | 0 | 0 |
| B | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| C | 1 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| D | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| E | 0 | 1 | 1 | 0 | 0 | 1 | 0 | 0 |
| F | 0 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| G | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |
| H | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Lists contain 18 adjacency entries. Matrix has 8 × 8 = 64 cells and $A = A^T$.

Each undirected edge contributes two list entries, for $2|E| = 18$ total. A directed arc $u \to v$ appears only in `Adj[u]`, giving $|E|$ total entries.

# Edge lookup and neighbor enumeration have different costs

`Adj[u]` lists u's neighbors; $M[u, v] = 1$ iff uv is an edge. Here the matrix is symmetric with a zero diagonal. Let $n = |V|$ and $m = |E|$. Use ordinary unsorted lists and a directly indexed matrix.

| Operation | Adjacency lists | Adjacency matrix |
|---|---|---|
| Storage | $\Theta(n + m)$ | $\Theta(n^2)$ |
| Test $u \to v$ | Worst-case $O(1 + \deg^+(u))$ | $\Theta(1)$ |
| Enumerate neighbors of u | $\Theta(1 + \deg^+(u))$ | $\Theta(n)$ |

For undirected graphs, use $\deg(u)$ instead of outdegree. The extra 1 covers an empty list. A directed adjacency matrix need not be symmetric.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Loops and parallel edges require an explicit convention

In a directed multigraph, store one list entry for each arc, including each parallel arc and each self-loop. For an undirected multigraph, use two endpoint incidences per edge. A self-loop then contributes two entries in the same list, preserving a total of $2m$.

A binary matrix stores adjacency but loses multiplicity. A count matrix can retain multiplicity. Separate edge identities or individual weights need more storage. If an undirected loop is stored only once instead, the list total is $2m - \ell$, where $\ell$ counts loops.

BFS enqueues a vertex only on its first discovery. Loops and parallel edges add scans but do not add duplicate vertex enqueues.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Color, distance, and parent record different facts

White means undiscovered. Gray means discovered with its neighbor scan unfinished. Black means its neighbor scan has finished. The only transitions are `WHITE → GRAY → BLACK`.

$d[v]$ estimates the edge distance from A and starts at $\infty$. $\pi[v]$ records the vertex that first discovers v and starts at NIL. Set `d[A]=0`, color A gray, and enqueue it.

At each while-condition test, Q contains exactly the gray vertices. A dequeued vertex remains gray during its scan, so this equality does not hold at every instruction. In the handwritten notes p34, DFS d is a discovery timestamp. Here BFS d is a distance.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Mark a vertex gray before enqueueing it

```
for v in V: color[v]=WHITE; d[v]=∞; π[v]=NIL
color[A]=GRAY; d[A]=0; Q=[A]
while Q is not empty:
    u=dequeue_front(Q)
    for v in Adj[u] in alphabetical order:
        if color[v]==WHITE:
            color[v]=GRAY; d[v]=d[u]+1; π[v]=u
            enqueue_back(Q,v)
    color[u]=BLACK
```

Queue operations must take O(1) time. Removing an array's first element by shifting all remaining elements does not satisfy that assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

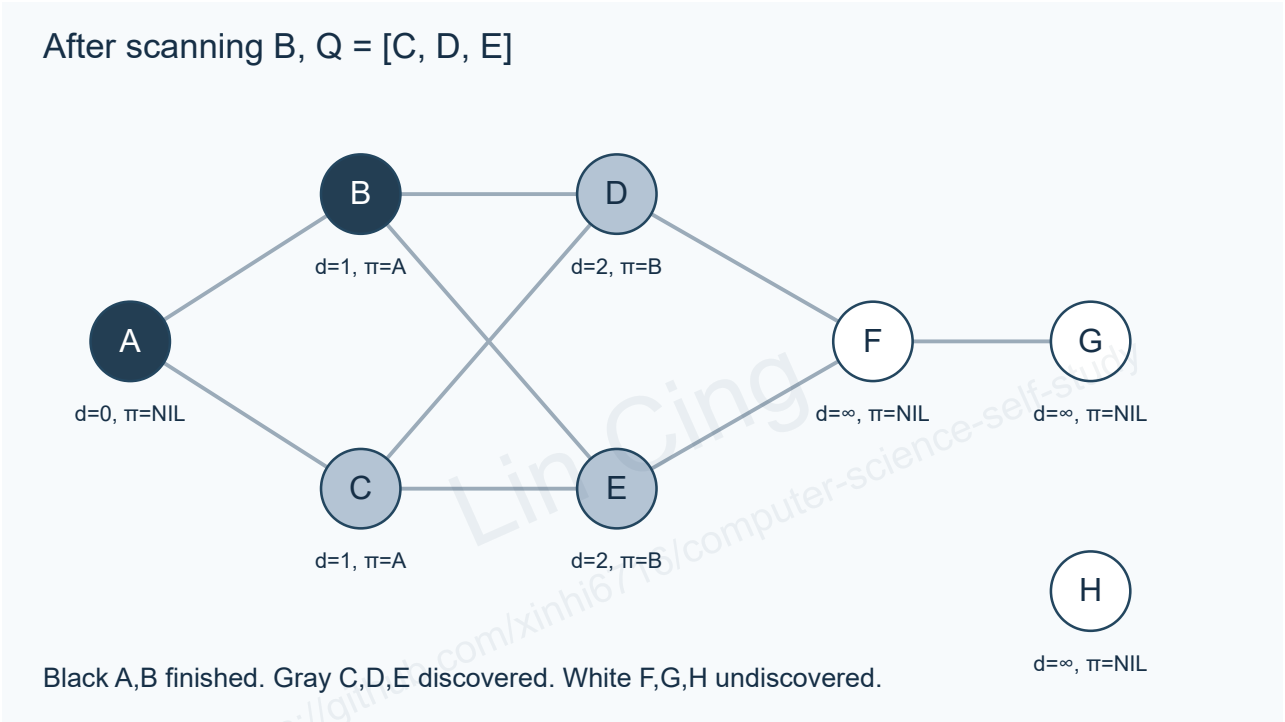
# The first two scans establish distances 1 and 2

Initially `Q=[A]`. Scanning A discovers B and C. Both turn gray, get distance 1, and get parent A. A turns black, leaving `Q=[B,C]`.

Dequeue B next. A is black, while D and E are white. Assign `d[D]=d[E]=2` and `π[D]=π[E]=B`. The completed scan leaves `Q=[C,D,E]`.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The first two scans establish distances 1 and 2 (continued)




C is already queued and must be processed before D and E.

# Record every completed scan in the queue trace

## Complete FIFO queue trace

| Scan finished | Newly discovered | Q front → back | Black vertices |
|---|---|---|---|
| init | A | [A] | ∅ |
| A | B,C | [B, C] | A |
| B | D,E | [C, D, E] | A,B |
| C | ∅ | [D, E] | A,B,C |
| D | F | [E, F] | A,B,C,D |
| E | ∅ | [F] | A,B,C,D,E |
| F | G | [G] | A,B,C,D,E,F |
| G | ∅ | [] | A,B,C,D,E,F,G |

Each row precedes the while test. Q contains exactly gray vertices. H stays white.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Read the queue trace step by step

When C is scanned, D and E are already gray. Their distances and parents stay unchanged. D first discovers F, assigning `d[F]=3, π[F]=D`. E then sees F gray. F discovers G, assigning `d[G]=4, π[G]=F`.

After scanning G, Q is empty. Each of the seven reachable vertices was enqueued once. H was never enqueued.

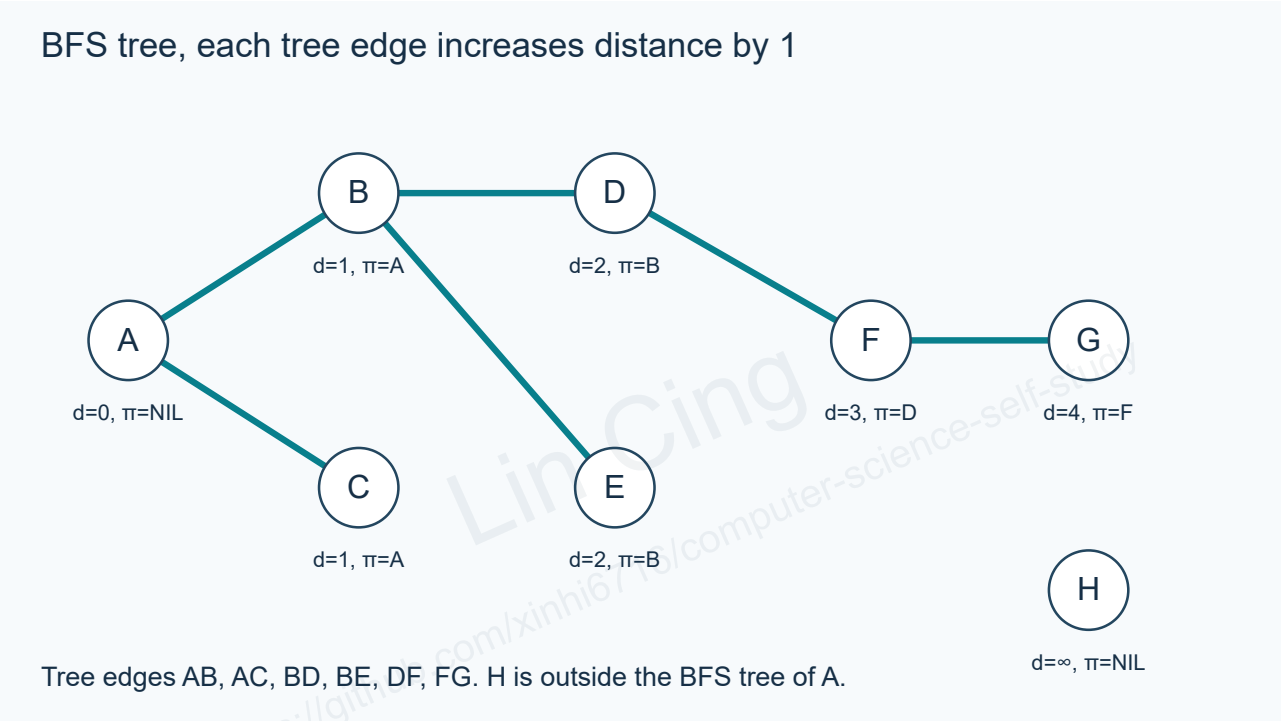
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The parent tree retains one shortest path

| v | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| d[v] | 0 | 1 | 1 | 2 | 2 | 3 | 4 | $\infty$ |
| $\pi$[v] | NIL | A | A | B | B | D | F | NIL |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The parent tree retains one shortest path (continued)




Following parents from G gives `G,F,D,B,A`. Reverse it to obtain `A,B,D,F,G`, a four-edge path. NIL can also mean the source. Check whether the target is A before interpreting NIL as unreachable.

# FIFO maintains nondecreasing distance order

When Q is nonempty, its distances are nondecreasing and the tail distance is at most one greater than the head distance. Only two adjacent layers can coexist in Q.

Dequeue removes the smallest value at the front. Scanning a vertex u with distance k assigns k+1 to new vertices and appends them at the back. Unprocessed vertices in the current layer stay ahead of the next layer.

After B, queued distances are `[1,2,2]`. After D, they are `[2,3]`. Every vertex of distance k is scanned before any vertex of distance k+1.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Shortest-path correctness needs both inequalities

Let $\delta(A, v)$ be the minimum edge count in the graph. For any discovered vertex, its parent chain is an actual path of length $d[v]$. Thus $\delta(A, v) \leq d[v]$.

Now induct on true distance k. A is correct for k=0. If $\delta(A, v) = k > 0$, its predecessor u on a shortest path has distance k−1. By induction u is labeled correctly and is eventually scanned.

If v is white when u scans it, assign $d[v] = k$. If v was already discovered, FIFO ensures the earlier discovering vertex had distance at most k−1, so $d[v] \leq k$. Combining both bounds gives $d[v] = k$. Every reachable vertex is therefore discovered with its minimum edge count.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Neighbor order changes the tree but not distances

Reverse every adjacency list. A discovers C before B. C discovers E and D, giving $\pi[D]=\pi[E]=C$. E is scanned before D, giving $\pi[F]=E$.

The dequeue order becomes $A,C,B,E,D,F,G$. The tree path to G becomes $A,C,E,F,G$. The distance vector stays $(0,1,1,2,2,3,4,\infty)$.

The correctness proof applies to both trees. The graph and source determine $\delta$. The parent records a choice among equally short paths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# One BFS does not traverse every component

After BFS from A, H remains white with $d[H] = \infty$. No path from A reaches H.

To find connected components in an undirected graph, initialize all vertices once. Then inspect each vertex and run BFS from every white root, assigning a new component label. Here the components are `{A,B,C,D,E,F,G}` and `{H}`.

Distances in the resulting forest are relative to each tree's root. In a directed graph, this outer loop produces a reachability search forest that depends on root order. It does not compute strongly connected components.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The representation determines the scan cost

Let $V_R$ be the vertices reachable from the source. Lists require $\Theta(n)$ initialization, one dequeue per reachable vertex, and one scan per associated adjacency entry. Total time is $\Theta(n + \sum_{u \in V_R} |Adj[u]|)$, bounded by $O(n + m)$.

A matrix scans a full row for each dequeued vertex, taking $\Theta(n + n|V_R|)$ time and $\Theta(n^2)$ in the worst case. Color, distance, parent, and queue require $O(n)$ extra space in either case, excluding the graph.

Here lists scan 18 entries. The matrix scans $7 \times 8 = 56$ cells. Both initialize 8 vertices. A full forest also scans H, raising matrix probes to 64 while list scans stay at 18.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# BFS minimizes the number of edges

If all edges have the same positive weight c, a minimum-edge path also minimizes total weight, with cost $c\,d[v]$. Unequal weights invalidate that conclusion.

For example, directed edges s→t of weight 10, s→x of weight 1, and x→t of weight 1 give a one-edge BFS path of cost 10. The minimum-weight path has two edges and cost 2.

The fixed graph and five SVGs are original examples. CLRS § 22.1–22.2 supports the representations, algorithm, and proof. MIT04 L14 and MIT05 L17 pp43–46,59 explain unit weights and FIFO. The notes p40 use BFS to choose a minimum-edge augmenting path in Edmonds–Karp.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# DFS returns a forest and nested time intervals

Full DFS discovers and finishes every vertex exactly once. It records parent pointers, discovery times d, and finish times f. Tree edges give reachability witnesses; intervals encode ancestry. A DFS tree path need not be shortest.

Trace one cyclic directed graph, then delete two edges to obtain a DAG and verify topological orders.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fix the graph and both traversal orders

Use a finite directed graph with no parallel edges or self-loops. The outer loop uses A,B,C,D,E,F. Adjacency lists have this exact order.

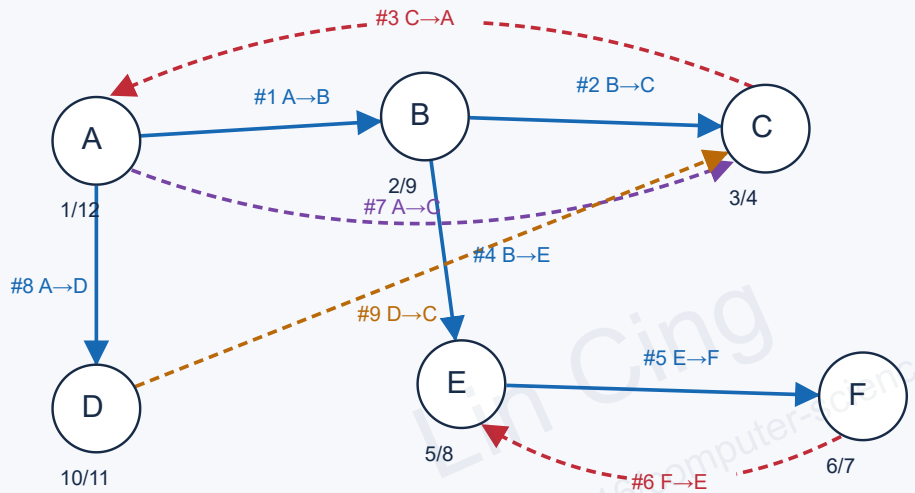
```
A:[B,C,D] B:[C,E] C:[A] D:[C] E:[F] F:[E]
```

Blue edges are tree edges, red back, purple forward, and orange cross. Types belong to this particular DFS execution; a different neighbor order may change them.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fix the graph and both traversal orders (continued)

Fixed order, edge scan numbers, and d/f




tree #1,#2,#4,#5,#8 back #3,#6 forward #7 cross #9

A:[B,C,D] B:[C,E] C:[A] D:[C] E:[F] F:[E]

Sources • MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22.

# Three colors describe a recursive call

WHITE means undiscovered. GRAY means discovered with an active call. BLACK means every outgoing edge has been scanned. The current source of an edge scan is the deepest GRAY vertex. All GRAY vertices form the ancestor chain on the call stack.

The clock advances only on discovery and finish, not on edge scans. Each vertex has $1 \le d[u] < f[u] \le 2|V|$. The $2|V|$ event times are distinct.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Full DFS and VISIT have different jobs

```
DFS(G):
  all colors = WHITE; all parents = NIL; time = 0
  for u in fixed vertex order:
    if color[u] == WHITE: VISIT(u)
VISIT(u):
  time += 1; d[u] = time; color[u] = GRAY
  for v in Adj[u] in fixed order:
    if color[v] == WHITE:
      parent[v] = u; VISIT(v)
  color[u] = BLACK; time += 1; f[u] = time
```

VISIT(u) explores vertices that can be discovered through white vertices from u. The outer loop covers all remaining vertices. A directed DFS forest is not a partition into strongly connected components. Edges may point to BLACK vertices in previous trees.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# The stack follows the event clock

| t | event | stack after |
|---|---|---|
| 1 | discover A | A |
| 2 | discover B | AB |
| 3 | discover C | ABC |
| 4 | finish C | AB |
| 5 | discover E | ABE |
| 6 | discover F | ABEF |
| 7 | finish F | ABE |
| 8 | finish E | AB |
| 9 | finish B | A |
| 10 | discover D | AD |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The stack follows the event clock (continued)

| t | event | stack after |
|---|---|---|
| 11 | finish D | A |
| 12 | finish A | ∅ |

Discovery pushes a call; finish pops it. At t=4, control returns to B and only then scans B→E. At t=9, control returns to A, which must still scan C and D. A finished vertex does not mean the entire algorithm has finished.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The stack follows the event clock (continued)

## All twelve events and call stacks

Each row shows the stack after the event, bottom to top.

| | | | | | |
|---|---|---|---|---|---|
| t= 1 discover A | A | | | | GRAY = A |
| t= 2 discover B | A | B | | | GRAY = AB |
| t= 3 discover C | A | B | C | | GRAY = ABC |
| t= 4 finish C | A | B | | | GRAY = AB |
| t= 5 discover E | A | B | E | | GRAY = ABE |
| t= 6 discover F | A | B | E | F | GRAY = ABEF |
| t= 7 finish F | A | B | E | | GRAY = ABE |
| t= 8 finish E | A | B | | | GRAY = AB |
| t= 9 finish B | A | | | | GRAY = A |
| t=10 discover D | A | D | | | GRAY = AD |
| t=11 finish D | A | | | | GRAY = A |
| t=12 finish A | ∅ | | | | GRAY = ∅ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Scan every edge, including visited targets

| # | edge | target | type | stack | clock |
|---|---|---|---|---|---|
| 1 | AB | W | tree | A | 1 |
| 2 | BC | W | tree | AB | 2 |
| 3 | CA | G | back | ABC | 3 |
| 4 | BE | W | tree | AB | 4 |
| 5 | EF | W | tree | ABE | 5 |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Scan the remaining edges and compare their types

| # | edge | target | type | stack | clock |
|---|---|---|---|---|---|
| 6 | FE | G | back | ABEF | 6 |
| 7 | AC | B | forward | A | 9 |
| 8 | AD | W | tree | A | 9 |
| 9 | DC | B | cross | AD | 10 |

The clock column is the last event time before the scan. W/G/B is the target color at that scan. A→C reaches a completed descendant and is forward. D→C also reaches BLACK, but C is not a descendant of D, so it is cross.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Align timestamps with the DFS tree

| vertex | d | f | parent |
|---|---|---|---|
| A | 1 | 12 | NIL |
| B | 2 | 9 | A |
| C | 3 | 4 | B |
| D | 10 | 11 | A |
| E | 5 | 8 | B |
| F | 6 | 7 | E |

Tree edges are A→B, B→C, B→E, E→F, A→D. Discovery order is A B C E F D. Finish order is C F E B D A.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Align timestamps with the DFS tree (continued)

## Call tree aligned with d/f intervals

Tree and intervals share vertex rows; x-axis is event time.




(A (B (C C) (E (F F) E) B) (D D) A)

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The parenthesis theorem rules out crossing intervals

For distinct u,v, [d[u],f[u]] and [d[v],f[v]] are disjoint or one strictly contains the other. Strict containment holds exactly when the inner vertex is a proper descendant of the outer vertex.

Proof. Assume d[u]<d[v]. If f[u]<d[v], u has returned before v begins, so the intervals are disjoint. Otherwise v is discovered while u is GRAY. Its recursive call is nested inside u. The call for v must finish before control can finish u, giving d[u]<d[v]<f[v]<f[u]. Exchange u,v for the other discovery order.

This execution gives `(A (B (C C) (E (F F) E) B) (D D) A)`. The ordering d[u]<d[v]<f[u]<f[v] is impossible.

Sources • CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# White-path theorem requires a snapshot

At the instant u is discovered, v becomes a descendant of u if and only if a path u→…→v has every vertex after u WHITE at that instant. The start u has just become GRAY. Equivalently take the snapshot immediately before coloring u, when every path vertex is white. The zero-length path covers v=u.

Necessity follows from the tree path: every proper descendant is discovered later. For sufficiency, choose the first path vertex x that fails to become a descendant. Its predecessor w is u or a descendant. Scanning w→x discovers x immediately if white. Otherwise x was discovered after u and before w finished. In both cases x is discovered while u is active, so it is a descendant, a contradiction.

Reachability alone is insufficient. C→A→D exists, but at d[C]=3, A is GRAY; D does not become a descendant of C.

Sources • CLRS3-22 § § 22.3–22.4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Four edge types in directed graphs

| Type | Test during scan | Relation after DFS |
|---|---|---|
| tree | target WHITE | parent[v]=u |
| back | target GRAY | v is ancestor of u, including a self-loop |
| forward | target BLACK and d[u]<d[v] | proper descendant, non-tree edge |
| cross | target BLACK and d[v]<d[u] | neither is ancestor of the other |

For distinct vertices, tree and forward edges share d[u]<d[v]<f[v]<f[u]. Timestamps cannot distinguish them without parent information. Back edges have d[v]<d[u]<f[u]<f[v]; cross edges have d[v]<f[v]<d[u]<f[u].

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Undirected edges are tree or back

In a simple undirected graph, each physical edge appears twice in adjacency lists. Classify it in the direction first encountered. Scanning the reverse entry of the parent edge does not create another back edge.

Proof. For {u,v}, assume d[u]<d[v]. The scan at u must reach v before u finishes, making v a descendant. If the first encounter is u to v, v is still white and the edge is tree. If first encountered from v to u, u is gray and the edge is back. No cross-subtree edge can remain.

For cycle detection skip the reverse entry of the same parent edge. In multigraphs use edge IDs: another parallel edge can make a length-2 cycle. A self-loop makes a length-1 cycle. State these conventions before applying the theorem.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A directed cycle exists exactly when a back edge exists

Use a finite directed graph, allowing self-loops. A cycle is a nonempty directed closed path with distinct internal vertices. Run full DFS.

A back edge u→v closes the tree path v→…→u into a cycle. A self-loop is already a cycle. Conversely, choose the earliest discovered vertex v on a cycle and let u be its predecessor on that cycle. At discovery of v, every other cycle vertex is white. The cycle provides a white path from v to u. The white-path theorem makes u a descendant of v, so u→v is a back edge.

Here C→A witnesses A→B→C→A, and F→E witnesses E→F→E. An edge to BLACK alone does not prove a cycle.

Sources • note-original; CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Delete two back edges to obtain a DAG

Let $G' = G - \{C \to A, F \to E\}$. Keep both traversal orders. All d/f values and tree edges stay unchanged. Reverse finish order is `A D B E F C`. Every retained edge points forward in that order, so it is a topological order.

The same order is invalid for G: $C \to A$ and $F \to E$ point backward. Every graph has a finish order, but only DAGs guarantee a topological order by reversing it.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

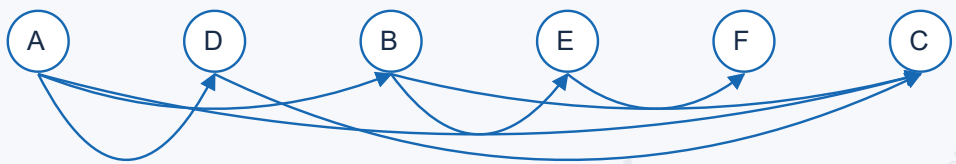
# Delete two back edges to obtain a DAG (continued)

Two topological orders of the same DAG

$G' = G - \{C \to A, F \to E\}$

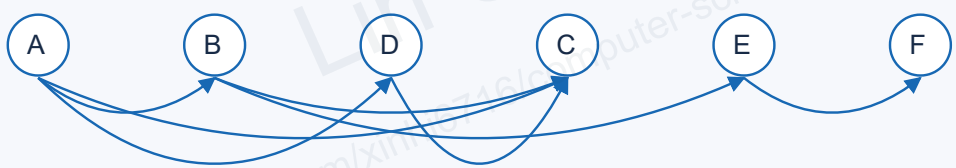
A D B E F C

DFS




A B D C E F

Kahn




Every arc points left to right; the valid order need not be unique.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Why reverse finish order works

For any DAG edge u→v, v cannot be GRAY at the scan, because that would give a back edge and a directed cycle. If v is WHITE, recursion finishes v before u. If v is BLACK, v is already finished and u is not. Thus f[u]>f[v] for every edge. Sorting by decreasing finish time places each source before its target.

Append each vertex when it finishes and reverse the list once, or insert at the head of a linked list. Repeated insertion at the front of an array can cost $\Theta(V^2)$. Detect a back edge and reject topological sorting on a cyclic input.

Sources • CLRS3-22 § § 22.3–22.4.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Kahn uses remaining indegrees on the same DAG

Repeatedly remove a vertex whose remaining indegree is zero and delete its outgoing edges. With alphabetical minimum selection, the execution is as follows.

| Removed | Available after deletion |
| --- | --- |
| A | B,D |
| B | D,E |
| D | C,E |
| C | E |
| E | F |
| F | ∅ |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Kahn uses remaining indegrees on the same DAG (continued)

The order A B D C E F differs from DFS but is valid. The invariant is that each selected vertex has no unprocessed predecessor. If vertices remain and every remaining indegree is positive, repeatedly following a predecessor must repeat a vertex, proving a cycle. FIFO gives $\Theta(V+E)$; alphabetical heap selection gives $O(E+V \log V)$.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Cost and self-study checks

With adjacency lists, full DFS takes $\Theta(V+E)$: one call per vertex and one scan per directed edge. Two scans per undirected edge preserve this bound. An adjacency matrix requires $\Theta(V^2)$ time. Colors, parents, timestamps, and the deepest recursive chain use $\Theta(V)$ additional space.

To reproduce recursion with an explicit stack, store frames `(u,next adjacency index)` and write f[u] only after the final edge. Pushing all neighbors onto a vertex-only stack need not reproduce this tree, finish times, or classification.

Exercises A07-D01 through D06 test timestamps, classification, theorem assumptions, undirected models, cycle proofs, and topological ordering.

Sources • MIT2004 pp.2–6; MIT2005 pp.2–5; CLRS3-22.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Connectivity questions have different outputs

An undirected connected component is a maximal set of mutually reachable vertices. A directed SCC requires reachability in both directions.

| Question | Output | Deletion test |
| --- | --- | --- |
| Connected components / SCCs | Vertex partition | None |
| Articulation points | Vertex set | Removing a vertex and incident edges increases component count |
| Bridges | Edge set | Removing an edge increases component count |
| Vertex-biconnected blocks | Edge groups, with shared cut vertices | No articulation inside a block |

Undirected inputs here are finite simple graphs, without loops or parallel edges. Roots and adjacency lists use alphabetical order. Discovery numbers start at 1. Sources [S1, S2].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A traversal forest labels undirected components

For each unlabelled vertex s, start DFS or BFS and assign one component_id to every vertex reached. If an unlabelled v were reachable from s, the first unlabelled vertex on a path would be discovered from its visited predecessor, a contradiction. Traversal cannot cross components because it follows existing edges. The outer loop must include isolated vertices. With adjacency lists, time is Θ(V+E) and auxiliary space is O(V). The two MIT BFS lectures support this traversal background; they do not supply SCC or articulation algorithms [S3, S4].

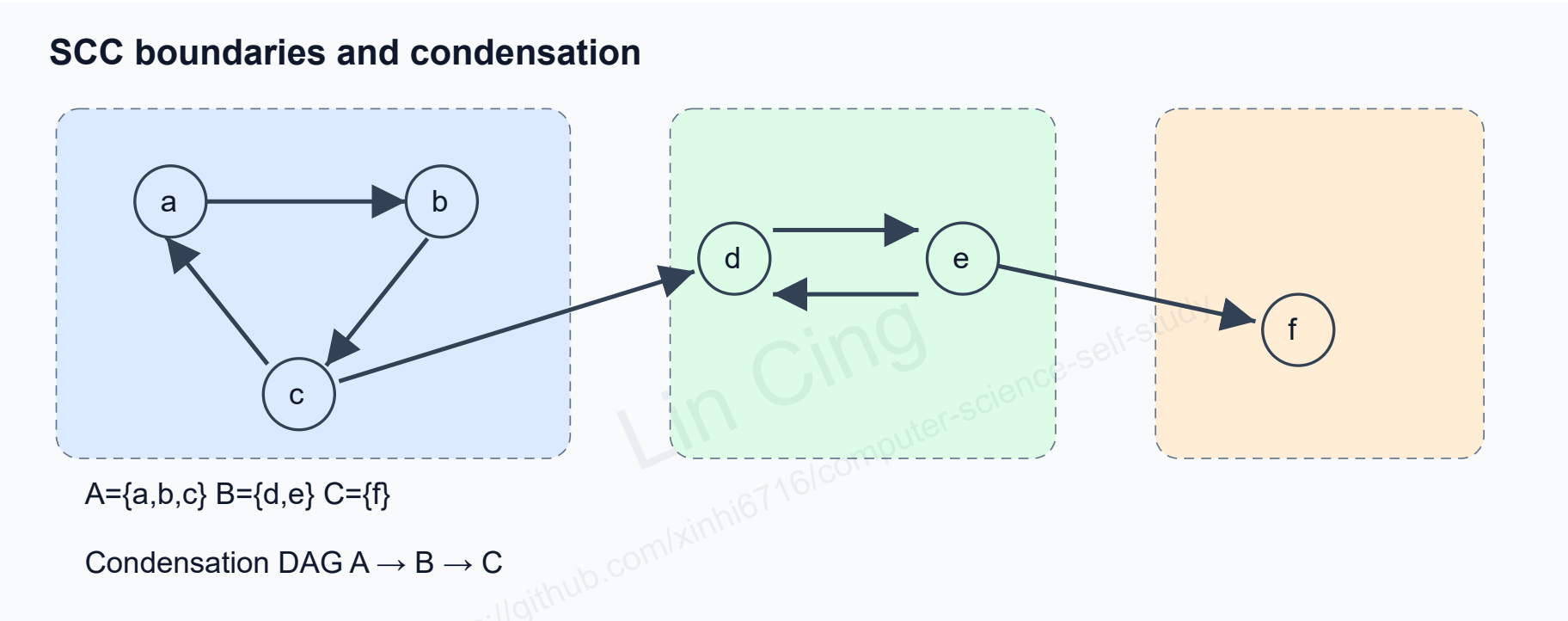
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Maximal SCCs produce a condensation DAG

Define u~v by mutual reachability. Zero-length paths give reflexivity; symmetry is explicit; path concatenation gives transitivity. SCCs are equivalence classes that partition V. Contract each SCC to a vertex, keep intercomponent arcs, discard loops, and optionally deduplicate arcs. This is the condensation graph. A directed cycle of two or more components would make all of them mutually reachable, contradicting maximality. The condensation is a DAG. A singleton SCC need not have a self-loop [S1 § 22.5].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Fix the example before tracing it




G has arcs ab, bc, ca, cd, de, ed, ef, where xy means x→y. The SCCs are A={a,b,c}, B={d,e}, C={f}; the condensation is A→B→C. One DFS from a reaches every vertex in G, so an original-graph DFS tree need not be one SCC.

# Kosaraju records finish times in the first pass

Run a complete DFS forest on G. Append each vertex to a finish list when its call returns. The finish list is f,e,d,c,b,a; reverse it for second-pass roots a,b,c,d,e,f.

| Vertex | Discovery d | Finish f | Parent |
|---|---|---|---|
| a | 1 | 12 | ∅ |
| b | 2 | 11 | a |
| c | 3 | 10 | b |
| d | 4 | 9 | c |
| e | 5 | 8 | d |
| f | 6 | 7 | e |

Increment the clock at discovery and finish. Distinguish the finish symbol f from vertex f [S1].

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Transpose preserves SCCs and reverses component arcs

$G^T$ replaces every u→v by v→u. A path u⇝v in G corresponds to v⇝u in $G^T$, so mutual-reachability classes stay unchanged. Transpose lists are a:[c], b:[a], c:[b], d:[c,e], e:[d], f:[e]. Its condensation is C→B→A. Create V empty lists, then append each reversed arc. This costs Θ(V+E). Clear visited for pass two, while retaining the first pass's finish order [S1].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# The finish-time lemma has two discovery cases

For distinct SCCs with an arc $C \to D$, define $F(C)=\max\{f(v):v\in C\}$ and $d(C)=\min\{d(v):v\in C\}$. We prove $F(C)>F(D)$.

1. If $d(C)<d(D)$, the first discovered $x$ in $C$ has all-white paths to every vertex of $C$ and, through the intercomponent arc, every vertex of $D$. They finish before $x$ returns. Thus $F(C)=f(x)>F(D)$.
2. If $d(D)<d(C)$, the first discovered $y$ in $D$ reaches all of $D$ but cannot reach $C$, since $D \rightsquigarrow C$ together with $C \to D$ would merge them. Every vertex of $D$ therefore finishes before any vertex of $C$ is discovered. Thus $F(D)<d(C)<F(C)$.

The lemma compares maximum finish times of components, not every pair of vertices across components [S1 Lemma 22.14].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Why each second-pass tree is exactly one SCC

Choose the unvisited root u of greatest first-pass finish time, in SCC C. Inductively, earlier trees are complete SCCs. Thus all of C is white, and F(C) is maximal among remaining components. An arc C→D in $G^T$ implies D→C in G. The lemma gives F(D)>F(C), so D has already been visited in pass two. Traversal from C cannot enter another unvisited component. Conversely C remains strongly connected in $G^T$ and is entirely white, so DFS from u reaches all of C. The tree contains exactly C. The empty forest is the induction base; repeat until all vertices are visited. Two DFS passes and transpose construction take Θ(V+E) time and O(V+E) extra space including $G^T$ [S1 Theorem 22.16].

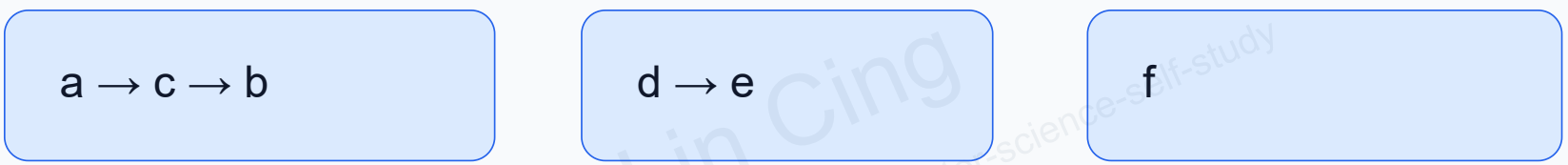
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Pass two shows roots, stacks, and skipped vertices

## Kosaraju directions and order

Pass 1 on G: finish order f,e,d,c,b,a

Reverse for root order a,b,c,d,e,f




Pass 2 on $G^T$: each box is one DFS tree and one SCC

Skip already visited root candidates b,c,e

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Pass two shows roots, stacks, and skipped vertices (continued)

| Root candidate | Deepest call stack in $G^T$ | Discovery order | Tree output |
|---|---|---|---|
| a | [a,c,b] | a,c,b | {a,b,c} |
| b,c | Already visited; skip | ∅ | ∅ |
| d | [d,e]; d→c is already black | d,e | {d,e} |
| e | Already visited; skip | ∅ | ∅ |
| f | [f]; f→e is already black | f | {f} |

The call stack empties after each tree. Do not reorder output using second-pass finish times. Reusing G with decreasing f incorrectly lets a absorb all six vertices.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

# Tarjan keeps a stack of unassigned vertices

disc[u] is the discovery index. low[u] is this recurrence's lowlink. S stores discovered vertices without an assigned SCC. onstack[u] means u∈S, and does not mean DFS-gray.

```
visit(u):
  disc[u] = low[u] = ++clock; push(S,u); onstack[u]=true
  for v in Adj[u]:
    if disc[v] is unset:
      visit(v); low[u]=min(low[u],low[v])
    else if onstack[v]: low[u]=min(low[u],disc[v])
  if low[u]==disc[u]:
    repeat: x=pop(S); onstack[x]=false; emit x
    until x==u
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The outer loop visits every undiscovered vertex. Ignore edges to assigned SCCs. For an already discovered stack target, use disc[v], not an arbitrary substitution of low[v]. Modern notation adapted from [S5].

# A complete Tarjan return and pop trace

### Tarjan stack and component boundaries

S is bottom to top; top is at the right

[a,b,c,d,e,f] → pop f

[a,b,c,d,e] → pop e,d

[a,b,c] → pop c,b,a

low = [1,1,1,4,4,6] SCC = {f} | {d,e} | {a,b,c}

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

S runs bottom to top. Discovery indices for a,b,c,d,e,f are 1,2,3,4,5,6.

# Tarjan discoveries and low updates

| Event | Update | s |
|---|---|---|
| Enter a | disc[a]=low[a]=1 | [a] |
| Enter b | disc[b]=low[b]=2 | [a,b] |
| Enter c | disc[c]=low[c]=3 | [a,b,c] |
| c→a | low[c]=1 | [a,b,c] |
| Enter d | disc[d]=low[d]=4 | [a,b,c,d] |
| Enter e | disc[e]=low[e]=5 | [a,b,c,d,e] |
| e→d | low[e]=4 | [a,b,c,d,e] |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Tarjan returns and emitted components

| Event | Update | S |
|---|---|---|
| Enter f | disc[f]=low[f]=6 | [a,b,c,d,e,f] |
| Return f, 6=6 | Pop f; emit {f} | [a,b,c,d,e] |
| e receives f, returns | low[e]=min(4,6)=4; no pop | [a,b,c,d,e] |
| d receives e, 4=4 | low[d]=4; pop e,d; emit {d,e} | [a,b,c] |
| c receives d, returns | low[c]=min(1,4)=1 | [a,b,c] |
| b receives c, returns | low[b]=1 | [a,b,c] |
| a receives b, 1=1 | low[a]=1; pop c,b,a; emit {a,b,c} | [] |

Final low is 1,1,1,4,4,6 and every onstack is false. After b returns it is no longer gray, but remains on S. The call stack and S differ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Separate directed lowlink from undirected low

| Item | Directed Tarjan SCC | Undirected cuts / bridges |
|---|---|---|
| Non-tree target | Must still be onstack | Ancestor, excluding reversed parent edge |
| Stored objects | Unassigned vertices | Edges for biconnected blocks |
| Boundary test | low[u]=disc[u] | low[v]≥disc[u] or > |
| Output | Vertex partition | Cut vertices, bridges, edge blocks |

A directed edge can reach a returned call still on S. For a→b,b→a,a→c,c→b, b's call returns before c→b is scanned; that edge must still update low[c]. Testing gray alone misclassifies c. If singleton x is emitted before visiting y with y→x, onstack[x] is false. That edge cannot merge the SCCs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Undirected low describes a subtree escape

For u, low[u] is the smallest discovery index reachable by descending zero or more tree edges inside u's subtree, then taking at most one back edge to an ancestor; disc[u] is included. Initialize low[u]=disc[u]. After tree child v returns, minimize with low[v]. For a nonparent edge to discovered ancestor w, minimize with disc[w]. Undirected DFS has no edges between separate DFS subtrees: the first subtree would discover the other through that edge. Thus escaping past u depends on a back edge to a proper ancestor of u [S1 Problem 22-2].

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Articulation tests distinguish roots from nonroots

A nonroot u is an articulation point iff some DFS child v has low[v]≥disc[u]. Then v's subtree cannot reach a proper ancestor of u, so deleting u separates it from the parent side. If every child subtree reaches a proper ancestor, those routes keep the remaining vertices connected. A root r is an articulation point iff it has at least two DFS-tree children. Separate root subtrees have no interconnecting edge. With one child, its remaining subtree stays connected; zero children also gives no articulation. Do not apply the nonroot test to a root. In a—b, low[b]≥disc[a], but deleting a does not increase component count [S1 22-2(a,b)].

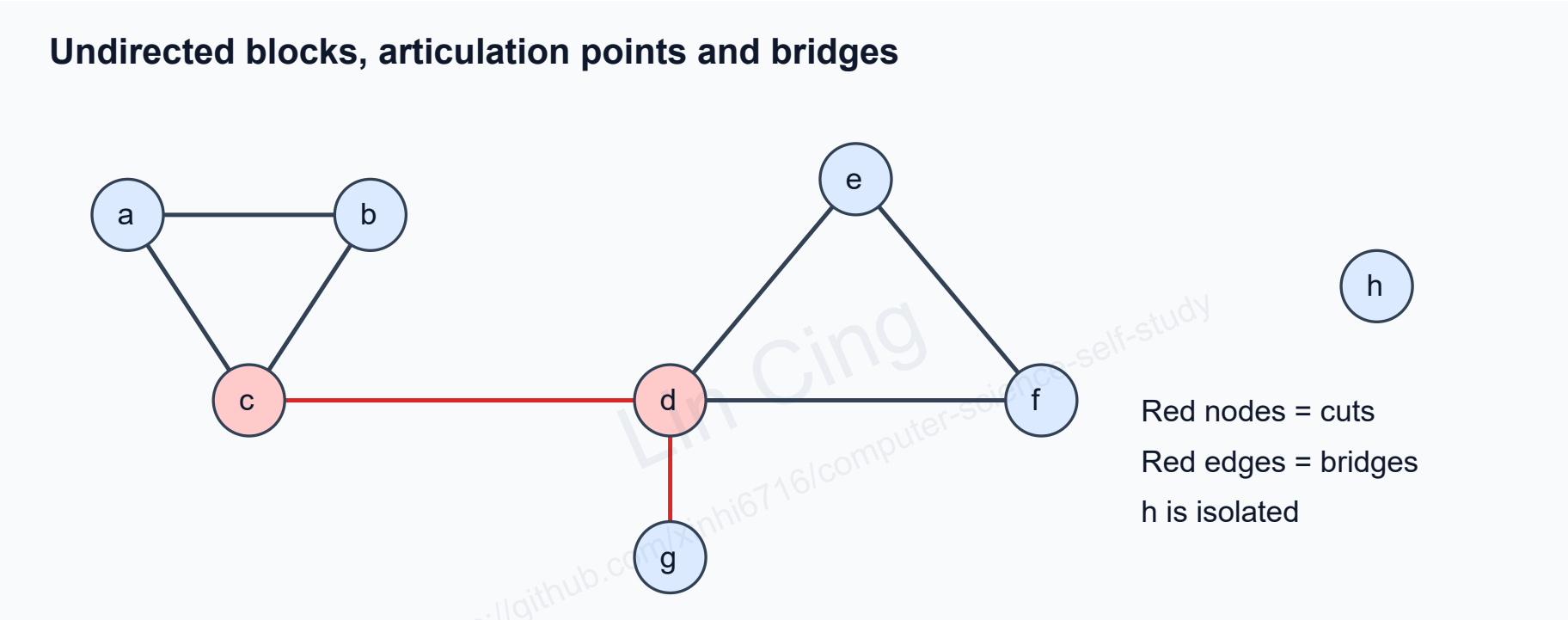
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# A bridge requires a strict inequality

A tree edge (u,v) is a bridge iff low[v]>disc[u]. Its subtree has no back edge to u or any earlier ancestor, so deleting the edge separates the sides. Equality low[v]=disc[u] gives a return to u and a cycle containing (u,v), so the edge is not a bridge. Deleting u also deletes that return endpoint, so nonroot u can still be an articulation point. In a simple undirected graph, every non-tree edge lies on a cycle and is not a bridge. For multigraphs, skip exactly the parent edge_id rather than all edges to the parent vertex. Two parallel edges are not bridges.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# An undirected example with complete disc / low values




Edges are ab,bc,ca,cd,de,ef,fd,dg; h is isolated. DFS uses alphabetical order.

# Undirected DFS disc / low: a through d

| Vertex | disc | low | Parent | DFS children |
|---|---|---|---|---|
| a | 1 | 1 | ∅ | b |
| b | 2 | 1 | a | c |
| c | 3 | 1 | b | d |
| d | 4 | 4 | c | e,g |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Undirected DFS disc / low: e through h and the tests

| Vertex | disc | low | Parent | DFS children |
|---|---|---|---|---|
| e | 5 | 4 | d | f |
| f | 6 | 4 | e | ∅ |
| g | 7 | 7 | d | ∅ |
| h | 8 | 8 | ∅ | ∅ |

Back edges c→a and f→d lower low[c] to 1 and low[f] to 4, then child returns propagate them. Articulation points are {c,d}; bridges are {cd,dg}. Root a has one child and is not a cut vertex. Vertex d is a nonroot despite its multiple children.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Biconnected blocks need an edge stack

Use the block convention: each bridge forms a two-vertex, one-edge block; isolated vertices receive singleton vertex blocks. Blocks with at least three vertices are maximal subgraphs without a cut vertex. This differs from the cycle-based bcc convention in CLRS 22-2, which partitions only nonbridge edges. Push each tree edge (u,v) before descent. Push each back edge to an ancestor once. When v returns with low[v]≥disc[u], repeatedly pop edges, including (u,v), then stop and emit that block. Apply this boundary to every root child independently of the root articulation test. Each edge is pushed and popped once, for Θ(V+E) time. Blocks partition edges; cut vertices can belong to several blocks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Every block boundary on the edge stack

Stacks run bottom to top. ca and fd are back edges; reversed parent edges are never pushed again.

| Event | Edge stack | Output |
|---|---|---|
| Push ab,bc,ca,cd,de,ef,fd in order | [ab,bc,ca,cd,de,ef,fd] | ∅ |
| f returns to e, 4<5 | Unchanged | ∅ |
| e returns to d, 4≥4 | [ab,bc,ca,cd] | Pop fd,ef,de |
| Push dg; g returns, 7≥4 | [ab,bc,ca,cd] | Pop dg |

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Return to the root and emit blocks

| Event | Edge stack | Output |
|---|---|---|
| d returns to c, $4 \geq 3$ | [ab,bc,ca] | Pop cd |
| c returns to b, $1 < 2$ | Unchanged | ∅ |
| b returns to a, $1 \geq 1$ | [] | Pop ca,bc,ab |
| Start new root h | [] | Isolated vertex {h} |

Blocks are {de,ef,fd}, {dg}, {cd}, {ab,bc,ca}, plus {h}. Popping only one edge, or stopping before de, splits the triangle incorrectly. The edge stack must be empty after each complete DFS tree.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## Use the formulas to answer failure questions

The example initially has two connected components: the main graph and {h}. Deleting c leaves {a,b}, {d,e,f,g}, {h}, for three components. Deleting d leaves {a,b,c}, {e,f}, {g}, {h}, for four. Deleting bridge cd produces three; deleting de still leaves two. A cut vertex can increase component count by more than one. A bridge models one failed link; a cut vertex models removal of a node and all incident links. SCCs instead express mutual reachability under direction constraints.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# Sources and their scope

S1 is CLRS third edition § 22.5, printed pp. 615–620; Problem 22-2, pp. 621–622, supplies cuts and bridges. S2 is the original handwritten note, PDF pp. 34–36, covering DFS, timestamps and edge types. Those reviewed pages do not contain SCC or articulation low formulas. S3 is MIT 2004 L14, PDF pp. 45–59; S4 is MIT 2005 L17, whose same range covers BFS. They support traversal background only. S5 is Tarjan's 1972 original paper; SCC notation is modernized to disc/low/onstack.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

# References and attribution

CLRS, Introduction to Algorithms, 3rd ed., §§22.1–22.5 and Problem 22-2. Bridges are included here as one-edge blocks. Original handwritten notes 演算法_wjungle, PDF pp. 34–36, 40, for DFS notation and BFS augmenting paths. MIT 6.046J Fall 2004, L13 pp. 2–6 and L14 pp. 43–59; Fall 2005, lec16 pp. 2–5 and lec17 pp. 43–59. Representation and BFS background. MIT 6.006 Fall 2011 Lecture 14 pp. 2–6; Spring 2020 Lecture 10 pp. 2–4. DFS and topological order. Robert Tarjan, “Depth-First Search and Linear Graph Algorithms,” SIAM J. Comput. 1(2), 146–160 (1972), doi:10.1137/0201010. SCCs.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study